

10 Proven Skills & MCPs Prompts

Copy any prompt, fill the [BRACKETS], send. Mined from 500+ hours of top Claude builders.

1

Install the Skill Creator

The Skill Creator turns skills from a black box into measurable software — it A/B tests with-skill vs without-skill so you actually know a skill helps.

Install the official Skill Creator plugin for me (run /plugin, search 'skill creator', install it). Once it's in, confirm it's ready — I'm going to use it to build, benchmark, and trigger-tune all my future skills.

2

Create a Skill the Right Way

The clarify-one-question-at-a-time pattern extracts everything from your head before any build, producing a reliable, reusable skill.

Use the skill-creator skill to build me a new skill called [SKILL NAME]. It should [WHAT IT DOES, INPUTS AND OUTPUTS]. First clarify your understanding by asking me questions one at a time until you're 95% confident, then discuss the data sources and tools, then build it. Save it to my global skills directory so it works across every project.

3

Skill-ify a Repeatable Workflow

Turns a repeated multi-sentence prompt into a one-command skill — the right mindset is 'standardize it, then skill-ify it.'

I keep doing this same multi-step workflow by hand: [DESCRIBE THE WORKFLOW]. First repeat the whole process back to me step by step so we agree on it. Then run /skill-creator and turn that exact process into its own skill so I can trigger it with one command in the future.

4

Benchmark a Skill

Evals turn 'use the skill more to make it better' into a measurable answer and tell you objectively when a skill should be kept, improved, or retired.

Benchmark my [SKILL NAME] skill: run the same test task with the skill loaded and without it, and show me a side-by-side of pass rate, total time, and total tokens so I can see the actual uplift. Then tell me honestly whether this skill is worth keeping, improving, or retiring.

5

Install an MCP Server Securely

Keeps API keys out of conversation history and public repos; the .env pattern is the standard secure MCP setup.

I want to connect to the [SERVICE] MCP server. Read its documentation here: [DOCS URL]. Install everything needed and create the .mcp.json config yourself. Do not put my API key in our chat – create a .env file and tell me exactly where to paste the key. When done, tell me to reload the window so you can use it.

6

Audit and Trim MCP Tools

Three MCP servers can eat ~66% of the context window before any work begins; trimming unused tools restores the budget and fights context rot.

Run /context and /mcp and show me exactly how many tokens each MCP server and its tools are costing me before I've even started working. Then disconnect any MCP servers I'm not using this session, and help me get my total enabled tool count under 50.

7

Prefer a CLI Over an MCP

CLIs are purpose-built for terminal agents and carry no config/overhead bloat — the Playwright CLI did the same job as its MCP for ~90k fewer tokens.

I need [BROWSER AUTOMATION / WEB SCRAPING / GOOGLE WORKSPACE ACCESS]. If a CLI tool exists for this, install the CLI instead of the MCP server – CLIs live in the terminal like you do and use far fewer tokens. Install the CLI and its matching skill together so you know how to drive it.

8

Adversarial Code Review With Codex

An LLM flatters its own code; a different model doing adversarial review catches critical bugs (soft-locks, save-rollback data loss) the author misses.

Install the Codex plugin for me and run codex setup. Then run an adversarial review on everything you've just built – pressure-test it across the 7 attack surfaces (auth, data loss, rollbacks, race conditions, degraded dependencies, version skew, observability gaps) and return severity-ranked findings. After, I'll have you implement the fixes.

9

Use Context7 for Current Docs

Models are trained on stale docs; Context7 keeps integrations current and prevents debugging loops caused by deprecated APIs.

Install the Context7 MCP server for me. From now on, whenever you integrate or recommend any library or API, use Context7 first so you pull the latest version-specific documentation and live code examples – I don't want you suggesting deprecated or renamed APIs from stale training data.

10

Reverse-Engineer a Site Into a Skill

Turns any admired website into a reusable, structured design system — far better than a lossy 60-70% HTML copy.

I want to turn a website I love into a reusable design skill. Point at [LIVE SITE URL], capture its layout, colors, fonts, spacing, and hover interactions, and build that into a skill. Then I'll be able to say 'build me a [TYPE OF SITE] in that style' and get a near one-shot starting point.